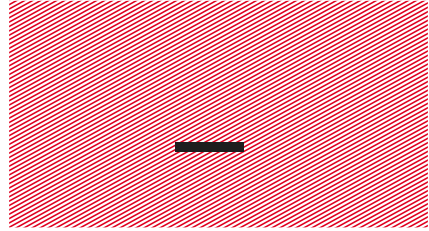


— IoT Devices

Les 6 – WiFi





WiFi

algemene info



WiFi algemene info

- **WiFi** (oorspronkelijk Wireless Fidelity) kan gebruikt worden om draadloos gegevens over te brengen via een 'computernetwerk'.
- Ieder apparaat krijgt een **IP-adres** (Internet Protocol) waardoor de gegevens naar de juiste locatie gestuurd kunnen worden.
- Het verzenden gebeurt dan meestal over **TCP** (Transmission Control Packet) of UDP (User Defined Packet).
- Daarbovenop kan je dan nog een applicatielaag leggen, bijvoorbeeld: **HTTP**.
- Zoek maar eens het **OSI-model** op. Daarin kan je de verschillende lagen terugvinden...

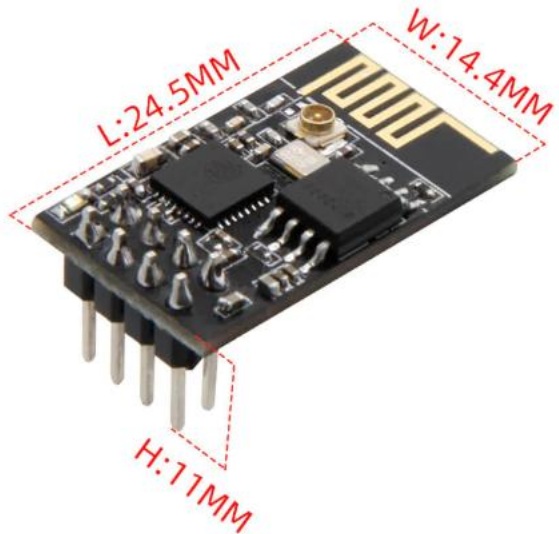
WiFi algemene info

Hieronder volgt een vereenvoudigde TCP/IP-stack, met enkele protocollen:

	Laag	Protocollen/systemen
7	Applicatie	HTTP , FTP , DNS , RTP <i>(routingprotocollen zoals OSPF en RIP maken deel uit van de applicatielaag, maar werken datastructuren in de netwerklaag bij)</i>
4	Transport	bijv. TCP , UDP , SCTP
3	Netwerk	TCP/IP gebruikt hiervoor het Internet Protocol (IP) <i>(protocollen zoals ICMP en IGMP worden bovenop IP gedraaid, maar kunnen beschouwd worden als deel uitmakend van de netwerklaag; ARP draait net onder IP, boven laag 2)</i>
2	Data Link	Ethernet , Token Ring
1	Fysiek	Fysieke media, en lijncodering , T1 , E1

WiFi algemene info

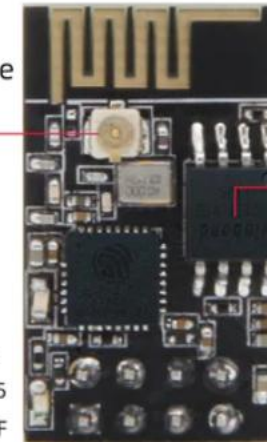
- In de les **IoT Devices** die wij zullen maken, gebruiken een WiFi-module: LILYGO T-01C3.
- Op die module is een Espressif **ESP32-C3** chip voorzien.



ESP32-C3 is a safe, stable, low-power, low-cost IoT chip, equipped with a RISC-V 32-bit single-core processor supporting 2.4 GHz Wi-Fi and Bluetooth 5 (LE)

External Antenna Base

--Security Mechanism
--Clock frequency 160 MHz
--Support Wi-Fi, Bluetooth 5
--Support Arduino、ESP-IDF



Flash

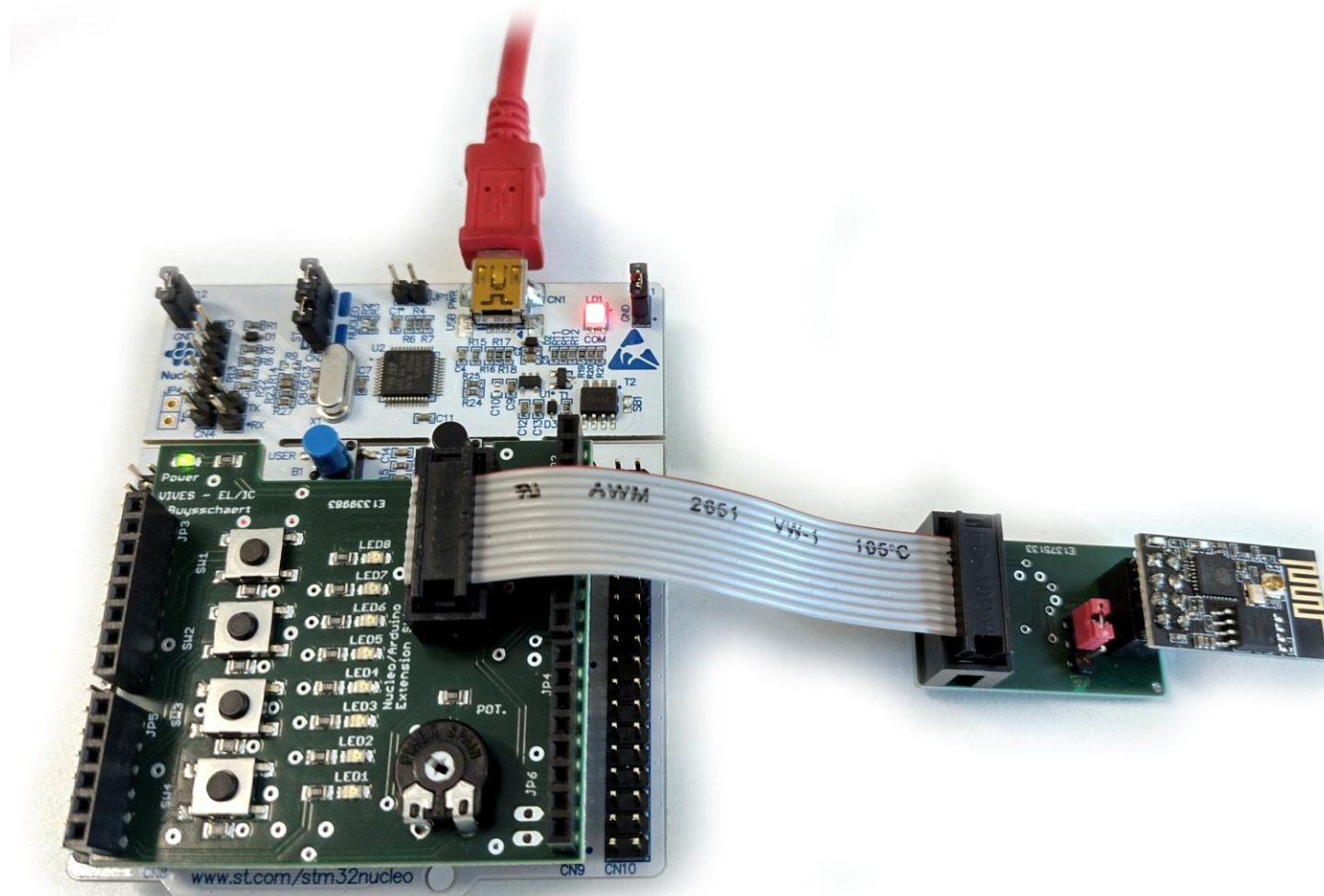
4M Byte/32M bit

GND IO2 IO9 RX
TX IO8 RST 3V3

- De LILYGO T-01C3 module kan je **herprogrammeren** (flash) met je eigen firmware.
- De fabrikant, Espressif, voorziet een **IDF** (IoT Development Framework) waarvan je kan starten.
- **Onze modules** zijn geprogrammeerd met een **AT-firmware** (zie verder)...
- De module is pin compatibel met de (oudere) ESP8266-module.

WiFi algemene info

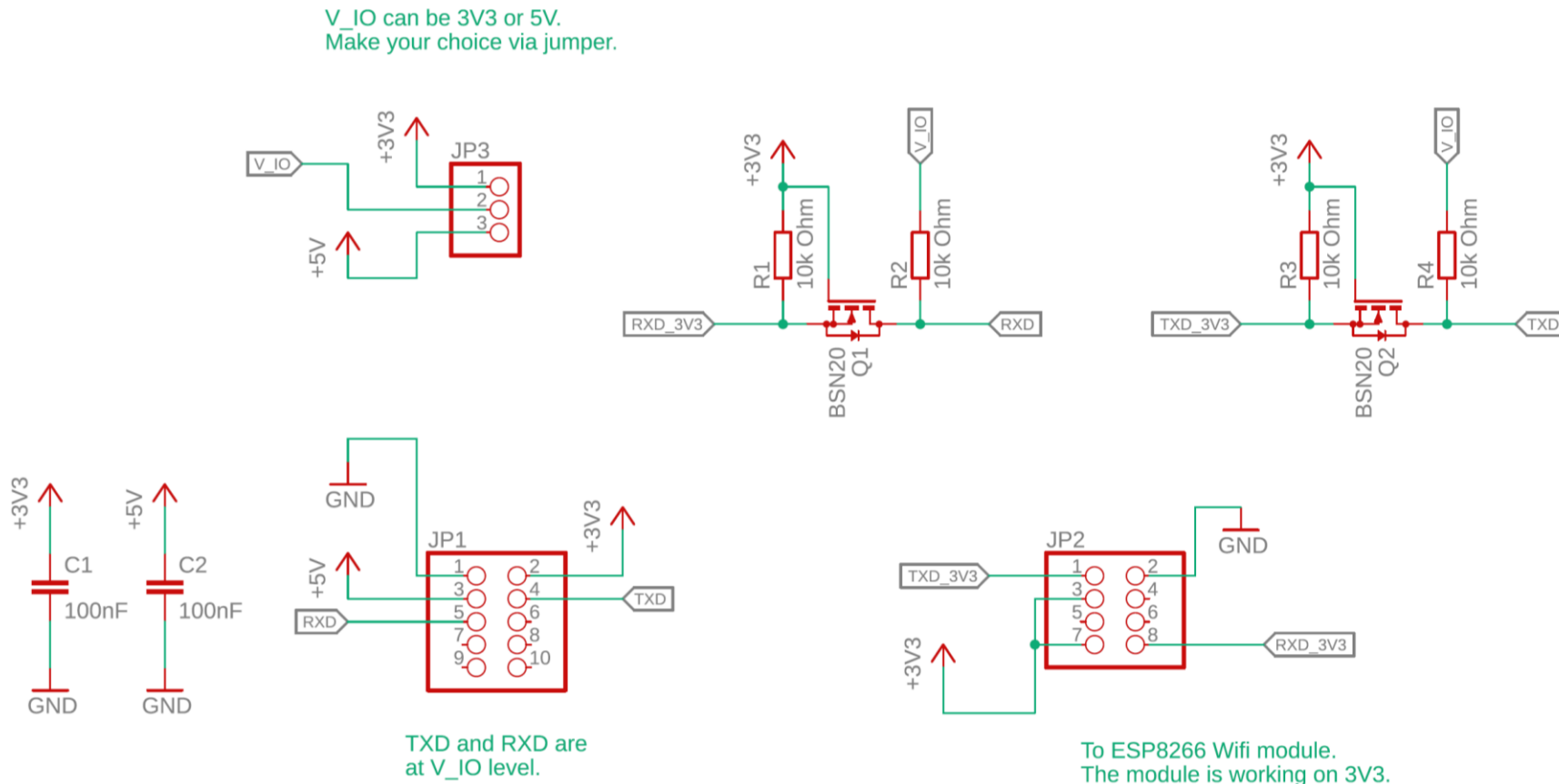
De **verbinding** kan gemaakt worden via een 10-pin flat cable.

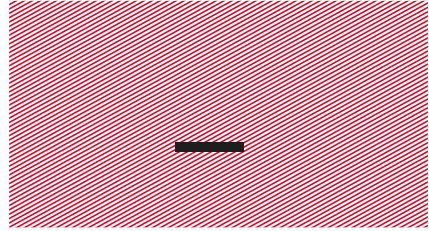


Bron: <https://www.lilygo.cc/products/t-01c3>

WiFi algemene info

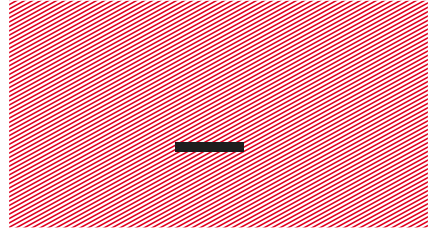
Om vlot te koppelen, werd er met een **level converter** gewerkt (optioneel). OPM: de converter is pin compatibel met de ESP8266.





Postman





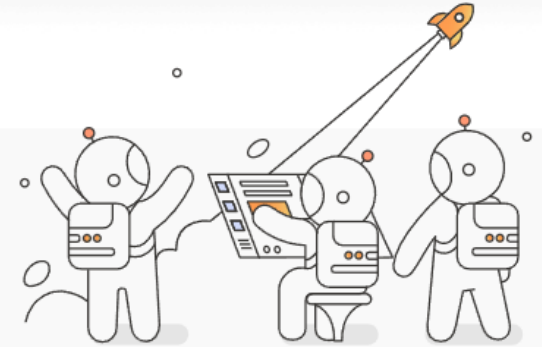
Postman

Wat?

Zie: postman.com

What is Postman?

Postman is an API platform for building and using APIs. Postman simplifies each step of the API lifecycle and streamlines collaboration so you can create better APIs—faster.



API Tools

A comprehensive set of tools that help accelerate the API Lifecycle - from design, testing, documentation, and mocking to discovery.



API Repository

Easily store, iterate and collaborate around all your API artifacts on one central platform used across teams.



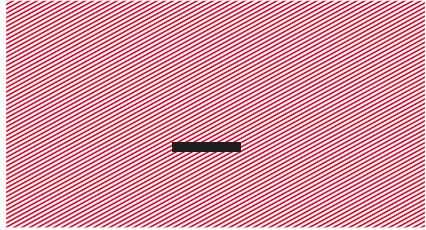
Workspaces

Organize your API work and collaborate with teammates across your organization or stakeholders across the world.



Governance

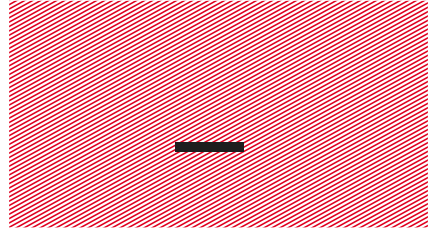
Improve the quality of APIs with governance rules that ensure APIs are designed, built, tested, and distributed meeting organizational standards.



Postman

Wat?

- Een applicatie dus, die je toelaat **dataverzoeken te versturen** over het netwerk.
- Wordt heel veel gebruikt om te **testen of je Web API werkt**.
- In ons geval kunnen we **web requests** versturen naar een server en het antwoord daarop bekijken.
- Dat kan handig zijn om **ervaring op te doen** en je IoT project voor te bereiden.



HTTP GET en POST

HTTP GET en POST

- Op de **rubu.be server** is er een pagina gemaakt met een heel eenvoudige Web API: <http://www.rubu.be/iot/esp32c3/index.php>.
- Je kan er een GET en POST request naar versturen.
- De API antwoordt met data of een error.

```
5 // Komt er een GET request aan met een correcte API Key?
6 if($_SERVER['REQUEST_METHOD'] === 'GET')
7 {
8     if(isset($_GET["apikey"]) && ($_GET["apikey"] == "api123"))
9         // Bij GET antwoorden met een getal voor op de LED's.
10        $data->leds = rand(0, 255);
11    else
12        $data->message = "GET API error.";
13 }
```

HTTP GET en POST

- Test het GET-verzoek met Postman...

The screenshot shows the Postman interface with a GET request configured. The URL is `https://www.rubu.be/iot/esp32c3/index.php?apikey=api123`, where the `apikey=api123` part is circled in red and labeled "Verzonden parameter (apikey)". The response status is `200 OK` with a response time of 154 ms and a size of 396 B. The response body is shown in JSON format, with the `"leds": 121` part circled in red and labeled "Ontvangen antwoord."

Verzonden parameter (apikey).

Ontvangen antwoord.

HTTP GET en POST

- Test het POST-verzoek met Postman...

Verzonden parameters (apikey en ad).

Overview POST post remo GET get remote GET GET HTTP POST POST HTT

Rubu ESP32 C3 IoT Devices / POST HTTP

POST <https://www.rubu.be/iot/esp32c3/index.php> Send

Params Auth Headers (9) Body Scripts Settings Cookies

x-www-form-urlencoded

	Key	Value	Description	Bulk Edit
☒	apikey	api123		
☒	ad	456		

Body ☒ JSON Preview Visualize

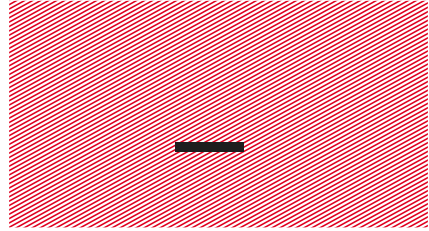
```
1 {  
2   "ad": "456"  
3 }
```

200 OK • 38 ms • 382 B • Save Response

Ontvangen antwoord.

HTTP GET en POST

- We kunnen dus via onze **laptop** informatie aanleveren aan een webserver en het antwoord daarop verwerken...
- Als we hetzelfde zouden doen met een WiFi-module, kunnen we een **IoT Device** maken.
- De voorgaande voorbeelden met GET en POST werden bewust gedemonstreerd via een gewone HTTP-verbinding. Dat is een onbeveiligde verbinding. Daardoor kan je gemakkelijker 'volgen' wat er gebeurt. **De volgende stap is via HTTPS werken.**



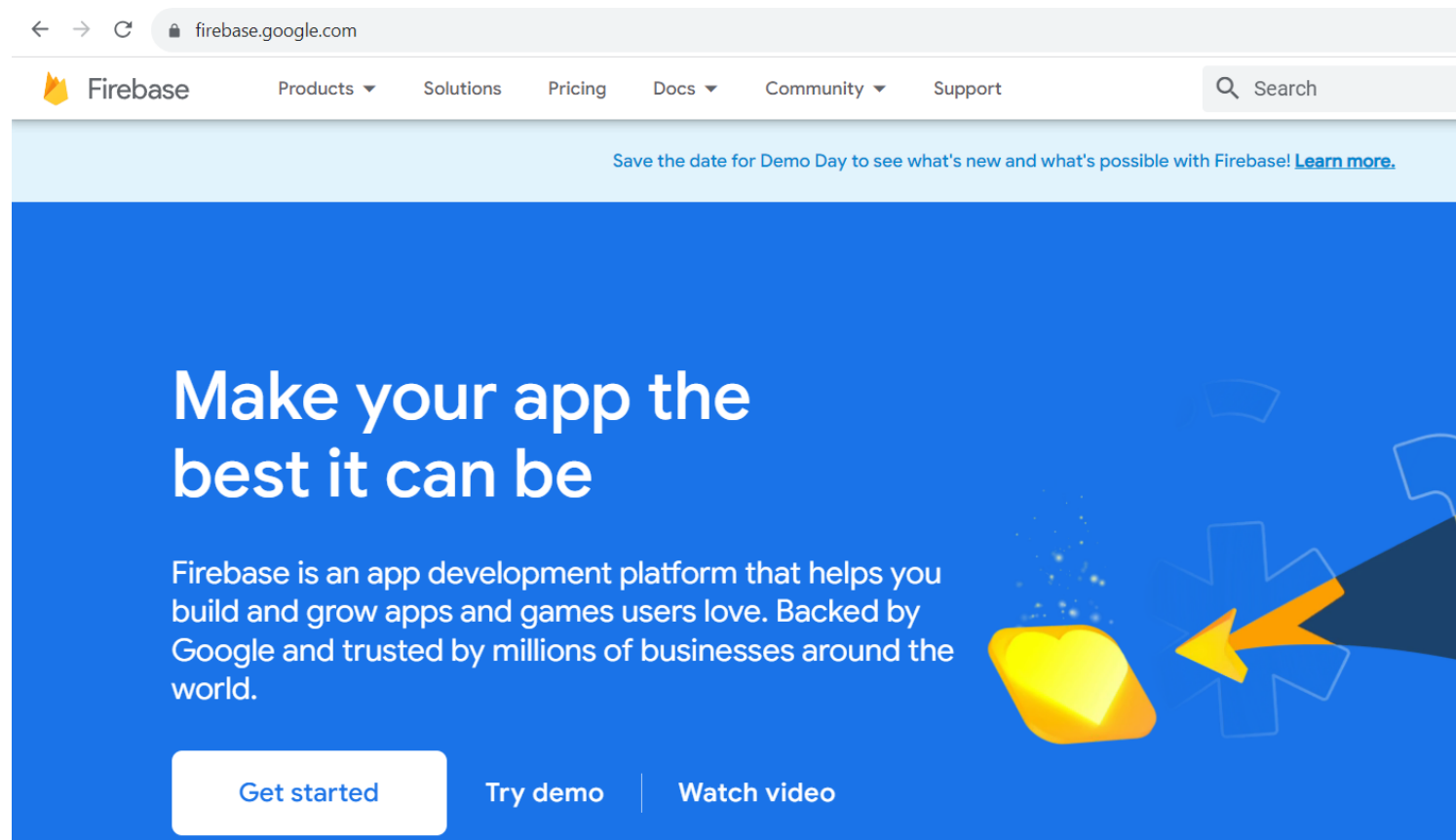
Firestore RTDB



—

Firestore RTDB

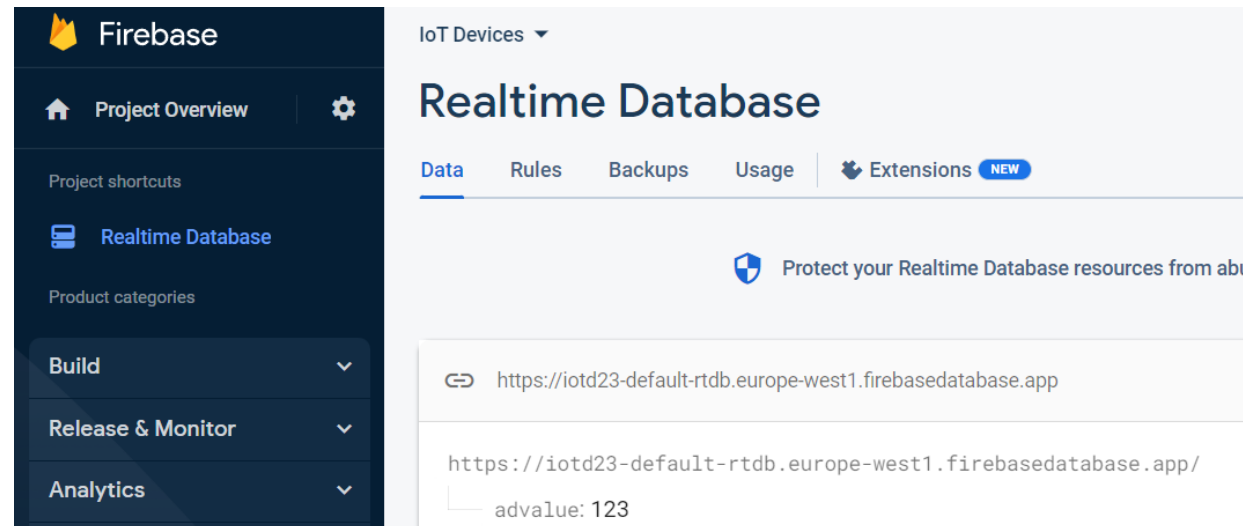
Firestore is een **ontwikkelaars platform** van Google, dat je toelaat om webruimte te reserveren, apps te hosten, databases op te zetten,...



—

Firestore RTDB

- Inloggen kan via: <https://console.firebase.google.com>.
- Je hebt uiteraard een **Google account** nodig daarvoor.
- Hoe je een Firebase project opzet, komt **later** nog aan bod.
- We zullen vooral de **Firestore RTDB** (Realtime Database) gebruiken om data op te slaan en op te vragen.



- Via de link <https://iotd23-default-rtdb.europe-west1.firebaseio.com> is er een **RTDB** gemaakt.
- Je kan er een GET en POST request naar versturen.
- De API antwoordt met data of een error.
- Deze API kan je ook weer testen met **Postman**.
- Merk wel op: je kan sowieso **lezen** (GET) uit de database, maar om te kunnen **schrijven/aan te passen** (PATCH) moet je authenticiseren.

Firestore RTDB

- Test het GET-verzoek met Postman...

Geen verzonden parameters nodig.

GET `https://iotd23-default-rtdb.europe-west1.firebaseio.com/advalue.json` Send

Params Authorization Headers (6) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (8) Test Results

Status: 200 OK Time: 24 ms Size: 294 B Save as Example

Pretty Raw Preview Visualize JSON

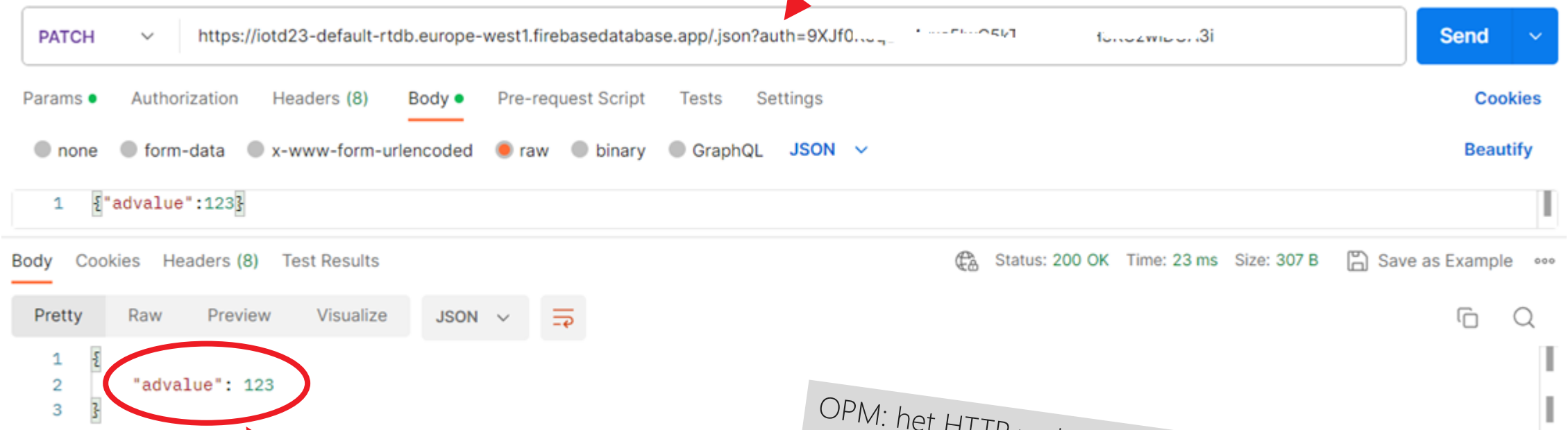
1 123

Ontvangen antwoord.

Firestore RTDB

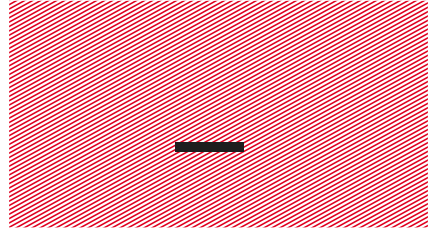
- Test het PATCH-verzoek met Postman...

De **authenticatie** wordt mee verzonden als parameter.



Ontvangen antwoord.

OPM: het HTTP verb PATCH, zorgt ervoor dat je een waarde kan **aanpassen** in de RTDB.



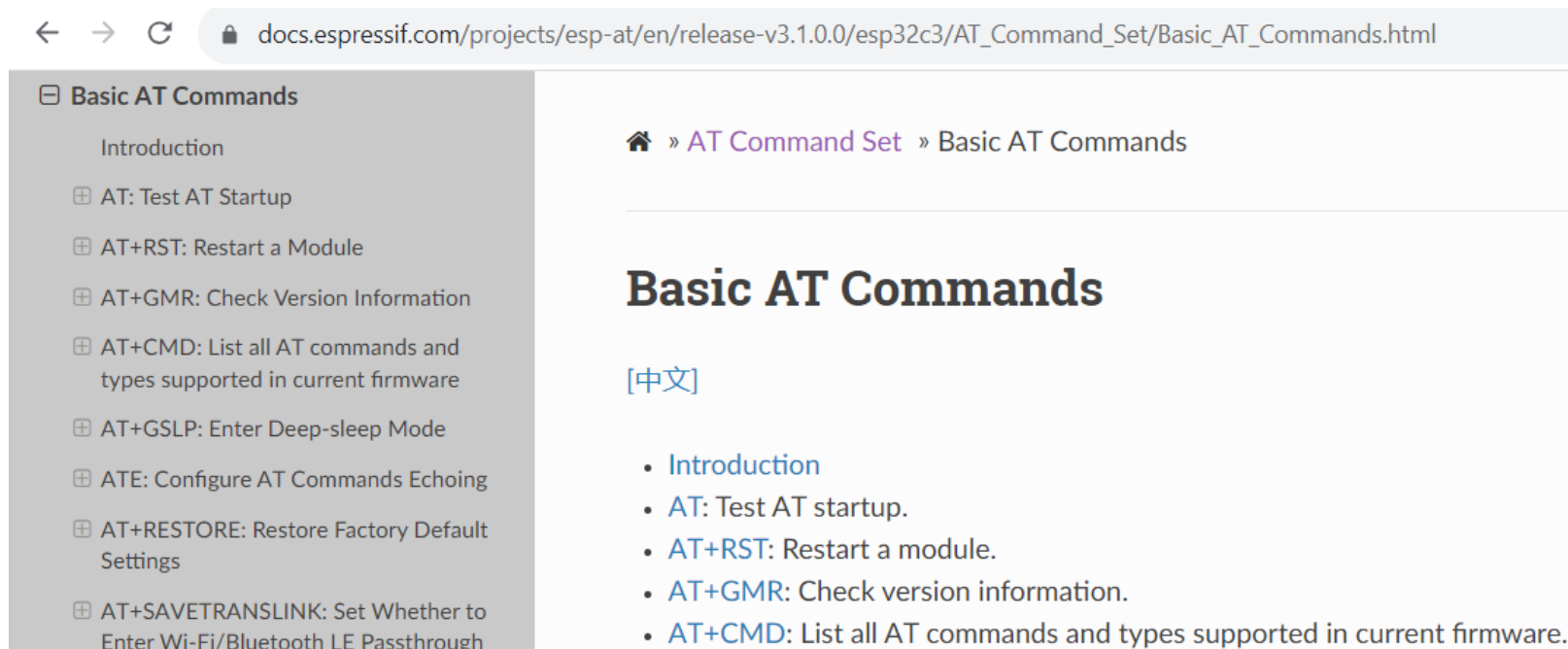
AT-commando's

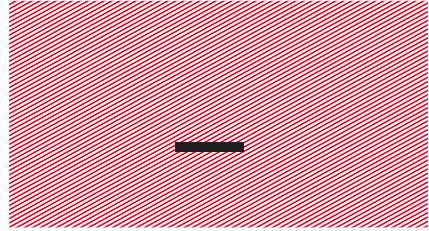


- Je kan apparaten koppelen via de **seriële poort** (UART).
- Over die UART kan je dan een protocol opzetten (= een manier van communiceren).
- Onze WiFi-module is ook zo gekoppeld met de Nulceo-L476RG.
- Communiceren zal gebeuren met **AT-commando's**...
- AT-commando's zijn 'stukjes tekst' die verstuurd worden over de UART met volgend formaat:
AT+COMMAND \r \n
- Bijvoorbeeld: *AT+RST \r \n* zorgt voor een reset van de module.

AT-commando's

- Een overzicht van **alle AT-commando's** kan je vinden op de website van de fabrikant:
https://docs.espressif.com/projects/esp-at/en/release-v3.1.0.0/esp32c3/AT_Command_Set/Basic_AT_Commands.html.





Circular buffer



Circular buffer

Een **typisch beeld** van de ontvangen data van een WiFi-module:

COM22 - PuTTY

```
AT+GMR
AT version:3.1.0.0-dev(s-7a56e30 - ESP32C3 - Apr 11 2023 09:48:35)
SDK version:v5.0-541-g885e501d99
compile time(dfa2d23):Apr 13 2023 16:01:17
Bin version:2.5.0(MINI-1)
```

```
OK
AT+CIPSTAMAC?
+CIPSTAMAC:"60:55:f9:7f:95:7c"
```

```
OK
AT+SYSSTORE=0
```

```
OK
AT+CWAUTOCONN=0
```

```
OK
AT+CWMODE=1,0
```

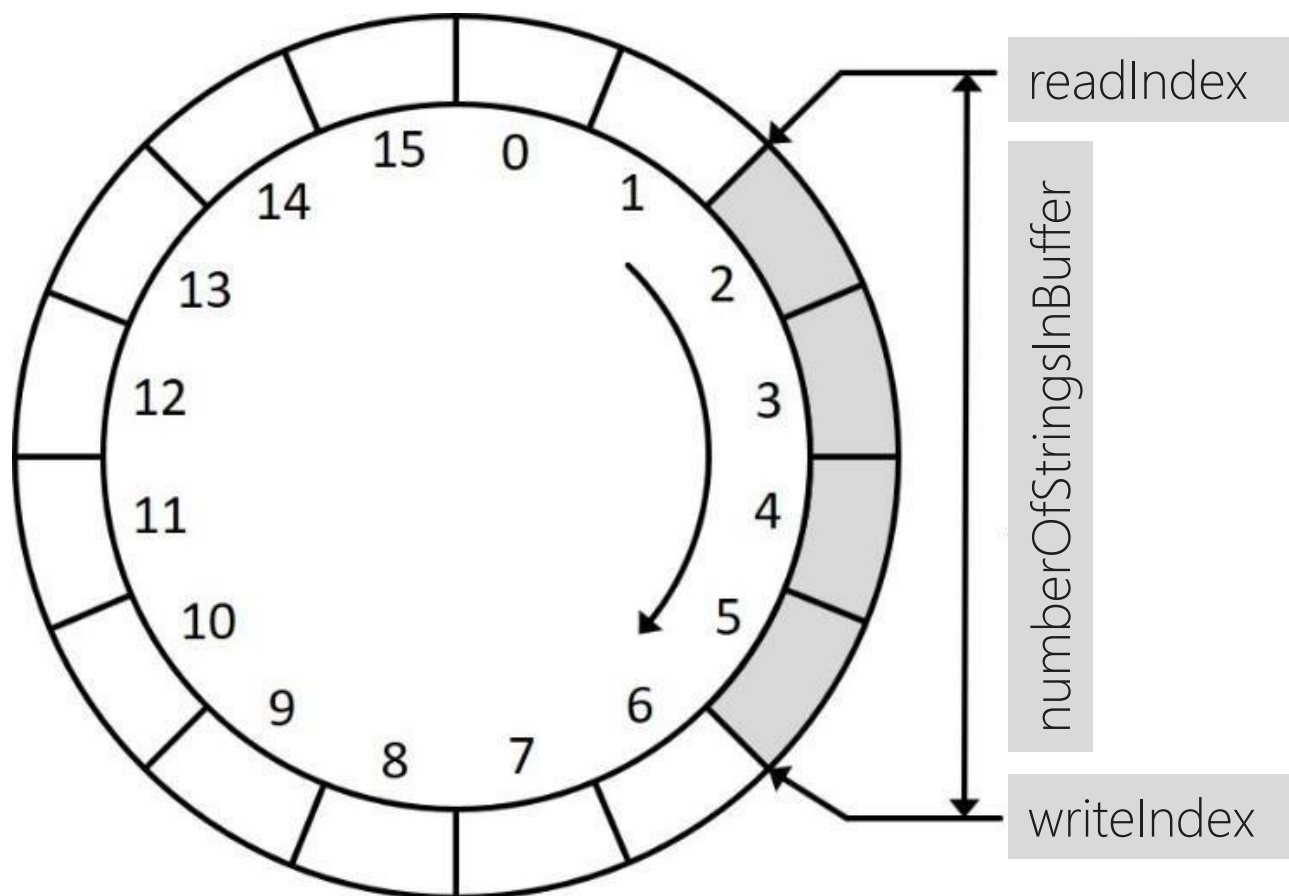
```
OK
AT+CWRECONNCFG=10,100
```

Hoe kan je dat **best verwerken**?

Circular buffer

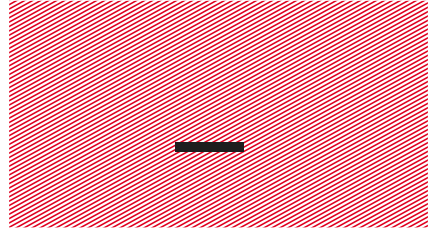
- Modules die via AT-commando's aangestuurd worden, verzenden vrij **veel data naar de microcontroller** via de UART.
- De kans is dus reëel dat je data bytes misloopt als je niet snel genoeg die bytes verwerkt...
- Daarom kan een **circulair buffer** heel handig zijn.
- Zo'n buffer **slaat heel snel alle aangeleverde bytes op**, die je dan later 'rustig aan' kan verwerken in de hoofdlus.
- Omdat het 'ciruclair' is, een soort **cirkel van bytes** dus, kan je 'oneindig lang' gegevens opslaan. Zolang je maar snel genoeg de opgeslagen data verwerkt.

Circular buffer



Bekijk zelf de **bestanden** 'circularbuffer.h' en 'circularbuffer.c' en zoek uit hoe alles werkt...

```
19 // Struct die een circulair buffer voorstelt.  
20 typedef struct  
21 {  
22     uint16_t readIndex;  
23     uint16_t writeIndex;  
24     uint16_t numberOfStringsInBuffer;  
25     uint16_t startIndexOfCurrentlyArrivingString;  
26     char bufferData[CIRCULAR_BUFFER_SIZE];  
27 }CircularBuffer;
```



ESP32-C3 demo



- Er is een **sjabloon** beschikbaar met als naam: "Nucleo-L476RG - WiFi ESP32 C3 Template - no credentials".
- Daarin zit een **raamwerk** om met de WiFi-modules aan de slag te gaan.
- De logingegevens voor de Firebase database zijn wel verwijderd. Later maak je zelf zo'n database aan en kan je jouw **eigen logingegevens toevoegen**.

ESP32-C3 demo

```
175  /* USER CODE BEGIN WHILE */
176  while (1)
177  {
178      // Watchdog timer resetten. Dit zorgt voor een herstart als de hoofdlus iets
179      // meer dan 6 seconden geblokkeerd is.
180      HAL_IWDG_Refresh(&hiwdg);
181
182      // AD-waarde inlezen.
183      adValue = (uint8_t)(GetAdValue(&hadc1) >> 4);
184
185      // Alle knoppen inlezen.
186      buttonsValue = 0;
187      if(SW1Active())
188          buttonsValue |= 0x01;
189      if(SW2Active())
190          buttonsValue |= 0x02;
191      if(SW3Active())
192          buttonsValue |= 0x04;
193      if(SW4Active())
194          buttonsValue |= 0x08;
195      if(UserButtonActive())
196          buttonsValue |= 0x10;
197
198      // LED's updaten.
199      ByteToLeds(leds);
```

OPM: gebruik de **watchdog timer** om automatisch te resetten als de hoofdlus te lang geblokkeerd staat.

Dat kan voorvallen als de AT-commando's niet tijdig/correct verwerkt geraken.

ESP32-C3 demo

```
294 // Mogelijks ontvangen TCP-, HTTP-, HTTPS-, of MQTT-berichten zoeken in het circulair buffer.
295 // De berichten zijn van de vorm: +IPD, of +MQTTSUBRECV:0,"rubu/esp8266",3,254.
296 while(circularBuffer.numberOfStringsInBuffer > 0)
297 {
298     // Eén string ophalen.
299     circularBufferActionResult = PopStringFromCircularBuffer(&circularBuffer, receivedText);
300     if(circularBufferActionResult == readStringSucceeded)
301     {
302         // HTTP(S)-ontvangst onderzoeken. Zoek achter "+IPD,". Dat geeft het begin aan van de ontvangen TCP-data.
303         if(strncmp("+IPD,", receivedText, 5) == 0)
304         {
305             // Zoek uit hoeveel bytes er ontvangen werden (optioneel).
306             numberOfReceivedBytes = (uint16_t)atoi(&receivedText[5]);
307             //sprintf(text, "Number of received bytes: %d.\r\n", numberOfReceivedBytes);
308             //StringToUsart2(text);
309
310             // Op het einde van de HTTP-headers is een lege lijn. Zoek ze. Sla dus alle HTTP-headers over.
311             // Daarna staat de gevraagde info, afkomstig van de server.
312             while(!LookForString1(&circularBuffer, "\r\n"));
313
314             // Even wachten, zodat de volgende lijn data kan binnenkomen.
315             while(circularBuffer.numberOfStringsInBuffer == 0);
316
317             // Lees die volgende lijn uit. Dat is de lijn met de ontvangen data vanop het netwerk.
318             circularBufferActionResult = PopStringFromCircularBuffer(&circularBuffer, receivedText);
```

Pinout & Configuration

Clock Configuration

Project Manager

Tools

Software Packs

Pinout

Pinout view

System view

Q



Categories A-Z

System Core >

Analog >

Timers >

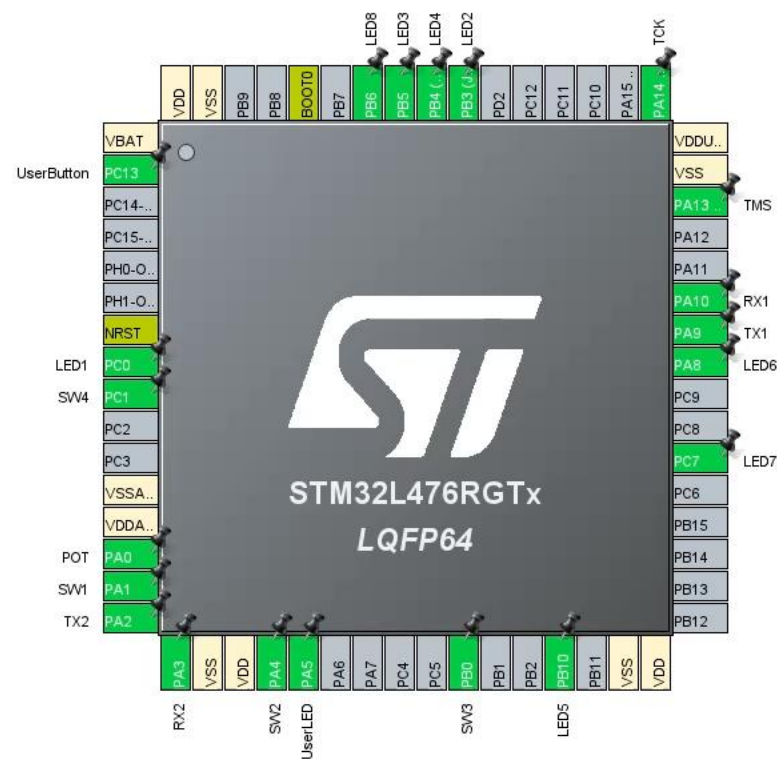
Connectivity >

Multimedia >

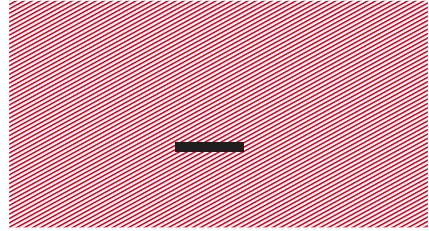
Security >

Computing >

Middleware and Software Pac... >



Unused GPIOs: 30 / 51



HTTP GET



HTTP GET

Doe nu, net als met Postman, een **GET request** naar rubu.be.

```
203 // HTTP GET voorbeeld (via rubu.be).
204 // Flankdetectie SW1.
205 if(SW1Active() && swlHelp == false)
206 {
207     swlHelp = true;
208
209     // GET parameters voorbereiden.
210     sprintf(httpParameters, "apikey=%s", HTTP_APIKEY);
211
212     // GET request versturen
213     HttpRequest(HTTP_HOST, HTTP_URL_GET, 80, GET, httpParameters);
214 }
215 if(!SW1Active())
216     swlHelp = false;
```

HTTP GET

Doe nu, net als met Postman, een GET request naar rubu.be.

```
AT+CIPSTART="TCP","rubu.be",80
CONNECT

OK
AT+CIPSEND=68

OK

>
Recv 68 bytes

SEND OK
HTTP-data sent:
GET /iot/esp32c3/index.php?apikey=api123 HTTP/1.1
Host: rubu.be
```

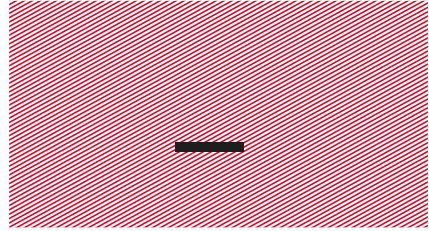
HTTP GET

Doe nu, net als met Postman, een GET request naar rubu.be.

```
+IPD,323:HTTP/1.1 200 OK
Date: Fri, 26 Sep 2025 06:15:32 GMT
Server: Apache
X-Powered-By: PHP/8.3.20
Access-Control-Allow-Origin: *
Content-Length: 12
Content-Type: application/json; charset=UTF-8
X-Varnish: 34194589539
Age: 0
Via: 1.1 webcache1 (Varnish/trunk)
Accept-Ranges: bytes
Connection: keep-alive

{"leds":169}
Decoded JSON: 169.
```

Het **antwoord** van de server is dus {"leds":169}. Die waarde wordt onderaan in de hoofdlus op de LED's gezet.



HTTP POST



Doe nu, net als met Postman, een **POST request** naar rubu.be.

```
234 // HTTP POST voorbeeld (via rubu.be).
235 // Flankdetectie SW2.
236 if(SW2Active() && sw2Help == false)
237 {
238     sw2Help = true;
239
240     // POST body voorbereiden.
241     sprintf(httpParameters, "apikey=%s&leds=%d", HTTP_APIKEY, adValue);
242
243     // POST request versturen.
244     HttpRequest(HTTP_HOST, HTTP_URL_POST, 80, POST, httpParameters);
245 }
246 if(!SW2Active())
247     sw2Help = false;
```



HTTP POST

Doe nu, net als met Postman, een POST request naar rubu.be.

```
SEND OK
HTTP-data sent:
POST /iot/esp32c3/index.php HTTP/1.1
Host: rubu.be
Content-Type: application/x-www-form-urlencoded
Content-Length: 22

apikey=api123&leds=254
```

HTTP POST

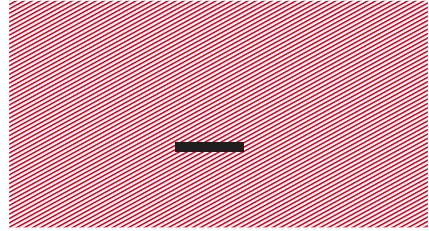
Doe nu, net als met Postman, een POST request naar rubu.be.

```
+IPD,301:HTTP/1.1 200 OK
Date: Fri, 26 Sep 2025 06:25:58 GMT
Server: Apache
X-Powered-By: PHP/8.3.20
Access-Control-Allow-Origin: *
Content-Length: 12
Content-Type: application/json; charset=UTF-8
X-Varnish: 34181252921
Age: 0
Via: 1.1 webcache1 (Varnish/trunk)
Connection: keep-alive

{"leds":254}
Decoded JSON: 254.
```

Het **antwoord** van de server is dus {"leds":254}. Dat komt omdat diezelfde 254 (op de vorige slide) verzonden werd als POST-info.

De ontvangen waarde wordt onderaan in de hoofdlus op de LED's gezet.



HTTPS GET



HTTPS GET

Doe nu, net als met Postman, een **HTTPS GET request** naar de Firebase RTDB.

```
235 // HTTPS GET voorbeeld (via Firebase).
236 // Flankdetectie SW3.
237 if(SW3Active() && sw3Help == false)
238 {
239     sw3Help = true;
240
241     // GET parameters voorbereiden.
242     sprintf(httpParameters, "auth=%s", HTTPS_DATABASE_SECRET);
243
244     // GET request versturen
245     HttpsRequest(HTTPS_HOST, HTTPS_URL_GET, 443, GET, httpParameters);
246 }
247 if(!SW3Active())
248     sw3Help = false;
```

HTTPS GET

Doe nu, net als met Postman, een HTTPS GET request naar de Firebase RTDB.

```
AT+CIPSTART="SSL","iotd23-default-rtdb.europe-west1.firebaseio.com",443
CONNECT

OK
AT+CIPSEND=137

OK

>
Recv 137 bytes

SEND OK
HTTPS-data sent:
GET /ledinfo.json?auth=9XJf0K8qDN4zxc5lwC5kT5Vo2ykvH3KGzw1DUA3i HTTP/1.1
Host: iotd23-default-rtdb.europe-west1.firebaseio.com
```

HTTPS GET

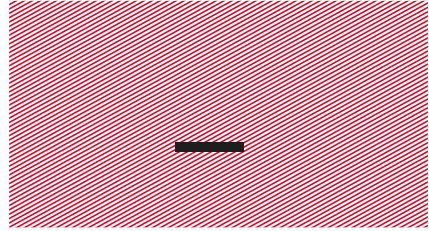
Doe nu, net als met Postman, een HTTPS GET request naar de Firebase RTDB.

```
+IPD,304:HTTP/1.1 200 OK
Server: nginx
Date: Fri, 26 Sep 2025 06:34:34 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 12
Connection: keep-alive
Access-Control-Allow-Origin: *
Cache-Control: no-cache
Strict-Transport-Security: max-age=31556926; includeSubDomains; preload

{"leds":123}
Decoded JSON: 123.
```

Het **antwoord** van de server is dus {"leds":123}. Dat komt omdat die waarde eerder al eens verzonden werd naar de RTDB.

De ontvangen waarde wordt onderaan in de hoofdlus op de LED's gezet.



HTTPS PATCH



HTTPS PATCH

Doe nu, net als met Postman, een **HTTPS PATCH request** naar de Firebase RTDB.

```
266 // HTTPS PATCH voorbeeld (via Firebase).
267 // Flankdetectie SW4.
268 if(SW4Active() && sw4Help == false)
269 {
270     sw4Help = true;
271
272     // Eerst PATCH data voorbereiden.
273     sprintf(httpParameters, "{\"leds\":\"%d\"", adValue);
274
275     // PATCH request versturen
276     sprintf(text, "%s?auth=%s", HTTPS_URL_PATCH, HTTPS_DATABASE_SECRET);
277     HttpsRequest(HTTPS_HOST, text, 443, PATCH, httpParameters);
278 }
279 if(!SW4Active())
280     sw4Help = false;
```

OPM: zie je dat we als info eigenlijk **JSON** verzenden...?

HTTPS PATCH

Doe nu, net als met Postman, een HTTPS PATCH request naar de Firebase RTDB.

```
+CIPSTATE:0,"SSL","34.107.226.223",443,64730,0
OK
AT+CIPSEND=218
OK
>
Recv 218 bytes

SEND OK
HTTPS-data sent:
PATCH /ledinfo.json?auth=vWNOKd1L1xAasqJEvV7NHTXUibu[REDACTED] HTTP/1.1
Host: iotd23-default-rtdb.europe-west1.firebaseio.com
Content-Type: application/json; charset=utf-8
Content-Length: 12

{"leds":145}
```



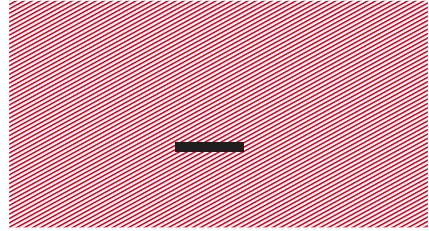
HTTPS PATCH

Doe nu, net als met Postman, een HTTPS PATCH request naar de Firebase RTDB.

Het **antwoord** van de server is dus ook JSON.

```
+IPD,304:HTTP/1.1 200 OK
Server: nginx
Date: Fri, 26 Sep 2025 06:57:12 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 12
Connection: keep-alive
Access-Control-Allow-Origin: *
Cache-Control: no-cache
Strict-Transport-Security: max-age=31556926; includeSubDomains; preload

{"leds":145}
Decoded JSON: 145.
```

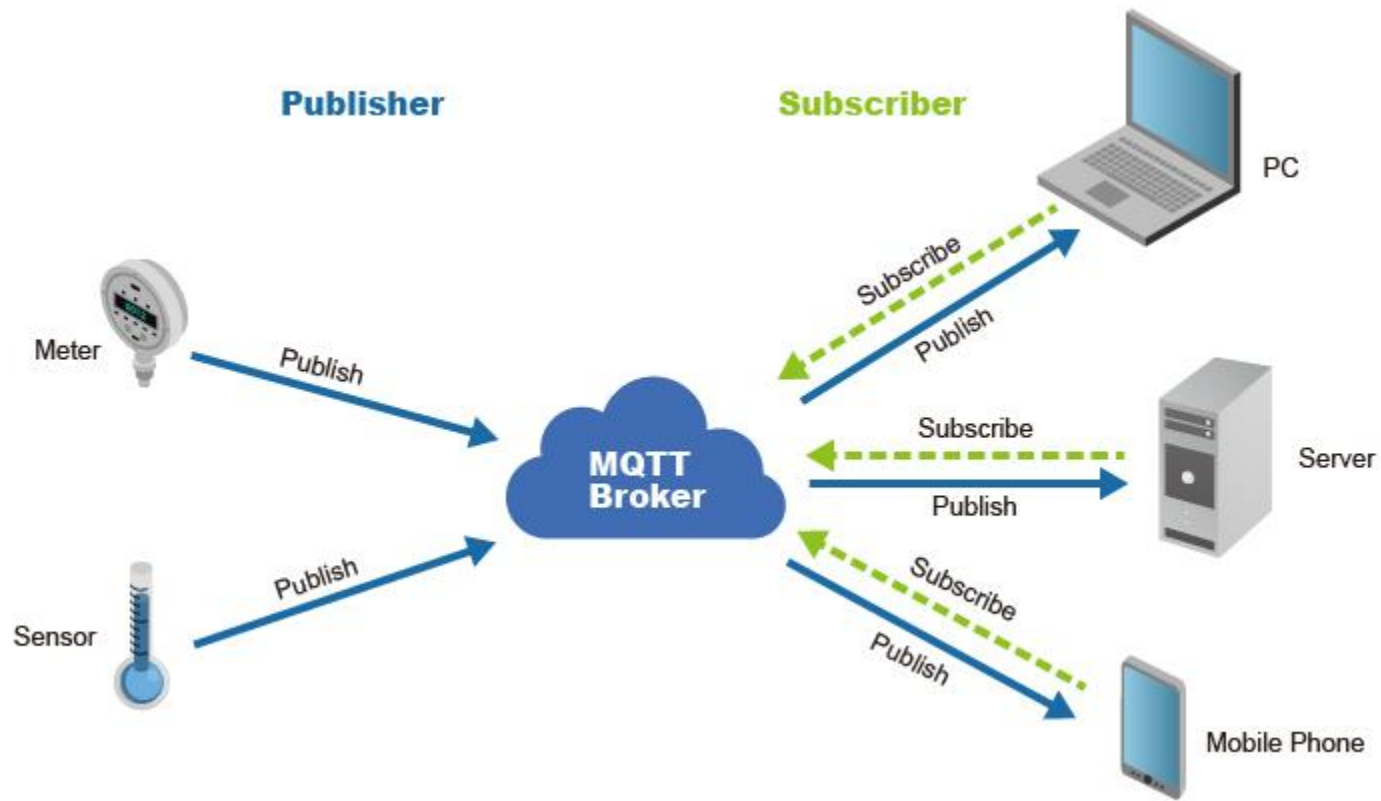


MQTT (over HTTP)



MQTT (over HTTP)

Herinner je uit het vak 'Microcontrollers'...



Bron: https://connect.oringnet.com/upload/media/Technology/MQTT/MQTT_1091217-01.jpg

MQTT (over HTTP)

Gebruik de User Button om automatisch **MQTT-berichten** te verzenden naar de mqtt.rubu.be broker.

```
284 // MQTT voorbeeld (via mqtt.rubu.be).
285 // Flankdetectie User Button.
286 if(UserButtonActive() && userButtonHelp == false)
287 {
288     userButtonHelp = true;
289
290     // Automatisch publish'n of niet?
291     timeBasedPublishing = !timeBasedPublishing;
292 }
293 if(!UserButtonActive())
294     userButtonHelp = false;
295
296 // Met behulp van de User Button het automatisch publish'n in- of uitschakelen.
297 // timerTicked wordt ingesteld in de stm32l4xx_it.c.
298 if(timeBasedPublishing && timerTicked)
299 {
300     // Timer verwerken.
301     timerTicked = false;
302
303     // MQTT-bericht publish'n. Gebruik later de AD-waarde om de LED's aan te sturen.
304     sprintf(text, "{\"leds:%d\\\"}", adValue);
305     MqttPublish(MQTT_PUB_TOPIC, text, 0, false);
306 }
```

MQTT (over HTTP)

Gebruik de User Button om automatisch MQTT-berichten te verzenden naar de test.mosquitto.org broker.

```
AT+MQTTPUBRAW=0,"iottest/1eds",11,0,0
```

```
OK
```

```
>+MQTTPUB:OK
```

```
MQTT-data published: {"1eds:44"}
```

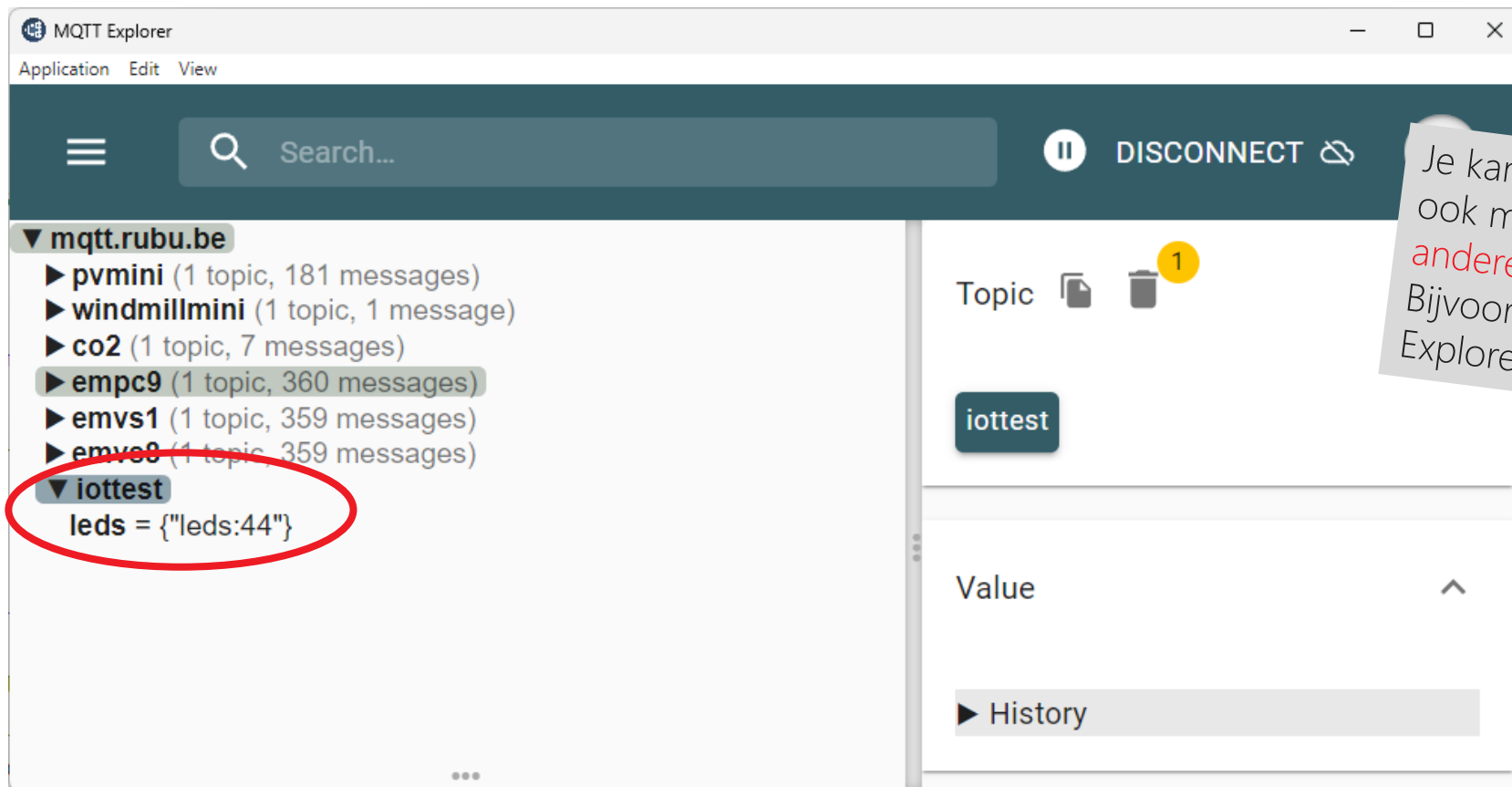
```
+MQTTSUBRECV:0,"iottest/1eds",11,{"1eds:44"}  
Decoded JSON: 44.
```

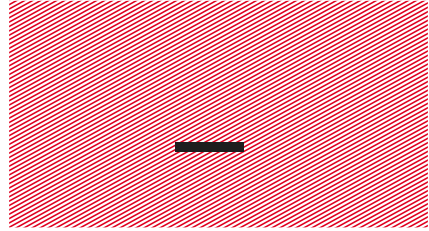
Het **antwoord** van de server is dus {"1eds":44}. Dat komt omdat diezelfde JSON net verzonden werd via MQTT naar de broker.

De ontvangen waarde wordt onderaan in de hoofdlus op de LED's gezet.

MQTT (over HTTP)

Gebruik de User Button om automatisch MQTT-berichten te verzenden naar de mqtt.rubu.be broker.





Theorieopdracht





Theorieopdracht

Maak gebruik van de demo code i.v.m. het gebruik van de WiFi-module om enkele vragen te beantwoorden.

De code heeft als naam: 'Nucleo-L476RG - WiFi ESP32 C3 Template – no credentials'.

Zie: "**Theorieopdracht 6 – WiFi**".